- **第一章复习**
- **LL（1）算法**

- **第一章复习**

第一章主要内容为对于编译器概念的引入。

（1）基本概念

第1章　引论

程序设计语言是向人以及计算机描述计算过程的记号。如我们所知，这个世界依赖于程序设计语言，因为在所有计算机上运行的所有软件都是用某种程序设计语言编写的。但是，在一个程序可以运行之前，它首先需要被翻译成一种能够被计算机执行的形式。

完成这项翻译工作的软件系统称为编译器（compiler）。

编译器(compiler)

读入以某种语言(源语言)编写的程序，输出等价的用另一种语言(目标语言）编写的程序，通常目标程序是可执行的。

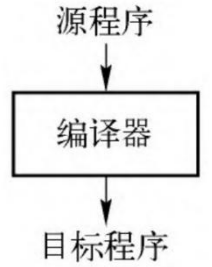


图1-1　一个编译器

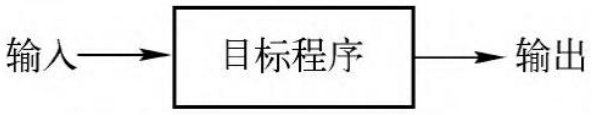


图1-2　运行目标程序

另一种处理程序设计语言的语言处理器是解释器，直接利用用户提供的输入，执行源程序中指定的操作，不生成目标程序，而是根据源程序的语义直接运行。

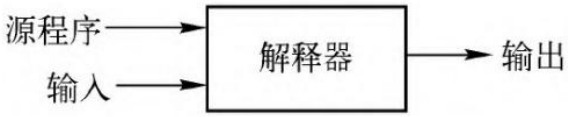


图1-3　一个解释器

Java 语言的处理结合了编译和解释（首先编译为字节码，再由机器解释执行）（一种优化的 Java 编译器是 JIT）。

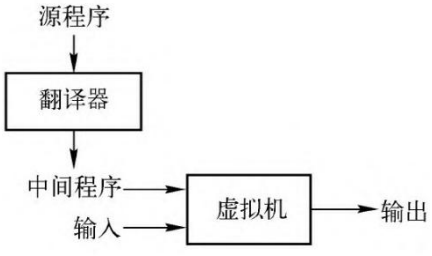


图1-4　一个混合编译器

为了更快地完成输入到输出的处理，有些被称为即时（just in time）编译器的Java编译器在运行中间程序处理输入的前一刻首先把字节码翻译成为机器语言，然后再执行程序。

（2）运行源程序的步骤

如图1-5所示，除了编译器之外，创建一个可执行的目标程序还需要一些其他程序。一个源程序可能被分割成为多个模块，并存放于独立的文件中。把源程序聚合在一起的任务有时会由一个被称为预处理器（preprocessor）的程序独立完成。预处理器还负责把那些称为宏的缩写形式转换为源语言的语句。

然后，将经过预处理的源程序作为输入传递给一个编译器。编译器可能产生一个汇编语言程序作为其输出，因为汇编语言比较容易输出和调试。接着，这个汇编语言程序由称为汇编器（assembler）的程序进行处理，并生成可重定位的机器代码。

大型程序经常被分成多个部分进行编译，因此，可重定位的机器代码有必要和其他可重定位的目标文件以及库文件连接到一起，形成真正在机器上运行的代码。一个文件中的代码可能指向另一个文件中的位置，而链接器（linker）能够解决外部内存地址的问题。最后，加载器（loader）把所有的可执行目标文件放到内存中执行。

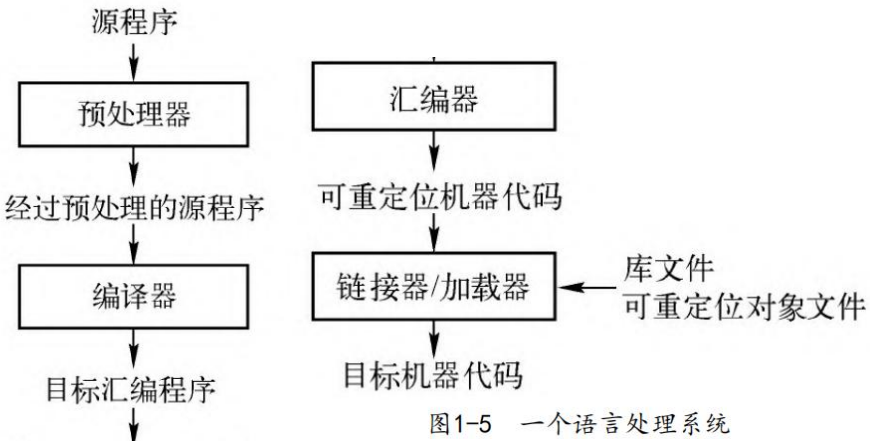


图1-5　一个语言处理系统

[GitHub - fool2fish/dragon-book-exercise-answers: Compilers Principles, Techniques, & Tools (purple dragon book) second edition exercise answers. 编译原理（紫龙书）第 2 版习题答案。]

（3）编译器结构

编译器可以分为分析部分和综合部分

分析(analysis) 部分/前端(frontend)

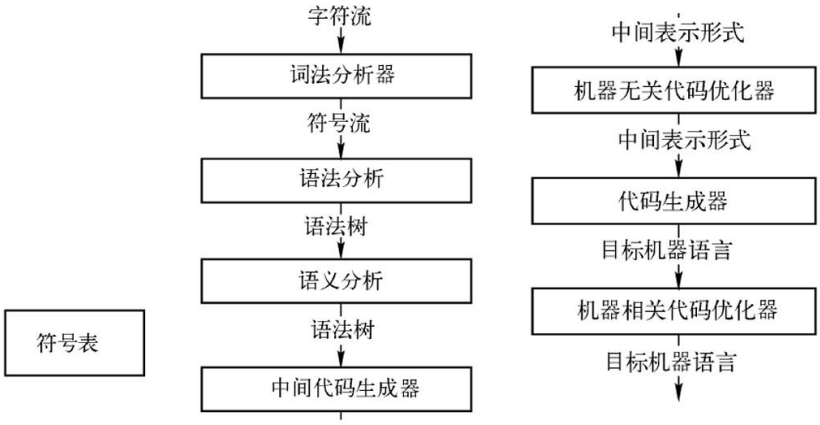
把源程序分解成组成要素，以及相应的语法结构，使用这个结构创建源程序的中间表示，同时收集和源程序相关的信息，存放到符号表。

综合(synthesis) 部分/后端(backend)

根据中间表示和符号表信息构造目标程序。

前端部分是机器无关的，后端部分是机器相关的。由于不同的硬件架构（例如 CPU 的指令集、寄存器数量、缓存结构等）各不相同，编译器的后端需要根据这些硬件特性生成和优化代码，以提高运行效率。这个阶段直接影响生成代码的执行性能，因此称为“机器相关”。

具体来说，如下图。



1.词法分析

编译器的第一个步骤称为词法分析（lexical analysis）或扫描（scanning）。词法分析器读入组成源程序的字符流，并且将它们组织成为有意义的词素（lexeme）的序列。对于每个词素，词法分析器产生如下形式的词法单元（token）作为输出：

〈token-name，attribute-value〉

分两部分，一是组织为词素序列（涉及到 buffering），二是将每个词素识别为词法单元。如课本 1.2.1 的这个例子。

```
position = initial + rate * 60
```

1）position是一个词素，被映射成词法单元〈**id**，1〉，其中**id**是表示标识符（identifier）的抽象符号，而1指向符号表中position对应的条目。一个标识符对应的符号表条目存放该标识符有关的信息，比如它的名字和类型。

对于标识符类型的词法单元，会组织一个符号表存放相关信息。

2. 语法分析（syntax analysis）

根据各个词法单元的第一个分量来创建树型的中间表示形式，通常是语法树中间表示形式指出了词法单元流的语法结构。第一个分量的意思是指如 id、*这种。对于语法错误的程序，报告错误信息；对于语法正确的程序，生成语法分析树(简称语法树）（通常并不真的产生这棵语法分析树）。

语法分析器和词法分析器及其他前端部分之间是一种协作的关系，如下图。

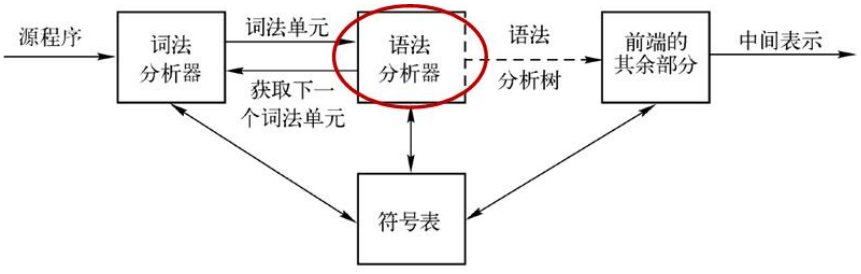


图 4-1　编译器模型中语法分析器的位置

3. 语义分析(semantic analysis)

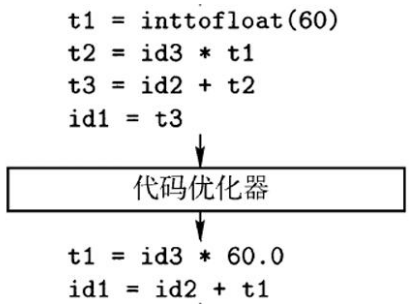
使用语法树和符号表中的信息，检查源程序是否满足语言定义的语义约束，同时收集类型信息，用于代码生成、类型检查、类型转换。这里书本主要讲的是类型检查这个功能。比如，这里的 60 被识别成浮点数类型。

中间代码生成
根据语义分析输出，生成类机器语言的中间表示
三地址代码(three-address code)每个指令最多包含三个运算分量，很容易生成机器语言指令。

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

关于三地址指令，有几点是值得专门指出的。首先，每个三地址赋值指令的右部最多只有一个运算符。因此这些指令确定了运算完成的顺序。在源程序1.1中，乘法应该在加法之前完成。第二，编译器应该生成一个临时名字以存放一个三地址指令计算得到的值。第三，有些三地址指令的运算分量的少于三个（比如上面的序列1.3中的第一个和最后一个指令）。

4. 中间代码优化
通过对中间代码的分析，改进中间代码的质量，更快、更短、能耗更低。这里书上指的是机器无关的代码优化。



5. 代码生成
把中间表示形式映射到目标语言，涉及到寄存器的分配和指令选择。
其实最后图中应该还有个机器相关的代码优化，不过没有专门开一个章节来讲。

- **LL（1）算法**

这是和语法分析有关的一个算法。
首先补充一些预备知识吧。
（1）文法，是上下文无关文法的简称（context free grammar，CFG），其实就是描述的程序设计语言的语法的意思，如何组织 if 语句，while 语句，诸如此类。
上下文无关文法 - 维基百科，自由的百科全书
他有四要素，书 2.2.1 小节也有关于这一点的概念介绍。

形式定义 [编辑]

上下文无关文法 G 是 4-元组：

$$G = (V, \Sigma, R, S)$$

这里的

1. V 是“非终结”符号或变量的有限集合。它们表示在句子中不同类型的短语或子句。
2. Σ 是“终结符”的有限集合，无交集于 V，它们构成了句子的实际内容。
3. S 是开始变量，用来表示整个句子（或程序）。它必须是 V 的元素。
4. R 是从 V 到 $(V \cup \Sigma)^*$ 的关系，使得 $\exists w \in (V \cup \Sigma)^* : (S, w) \in R$。

此外，R 是有限集合。R 的成员叫做文法的“规则”或“产生式”。星号表示Kleene星号运算。

表达式上 A -> α，其中 $A \in V$ ，$\alpha \in (V \cup \Sigma)^*$ 。

举例，如下图：

简单算术表达式的文法

- 终结符号：id, +, –, *, /, (,)
- 非终结符号：*expression*, *term*, *factor*
- 开始符号：*expression*
- 产生式集合

expression → *expression* + *term*
expression → *expression* – *term*
expression → *term*
term → *term* * *factor*　　*term* → *term* / *factor*
term → *factor*
factor → (*expression*)　　*factor* → id

（2）推导（derivation）

将待处理的串中的某个非终结符号替换为这个非终结符号的某个产生式的体 - 从开始符号出发，不断进行上面的替换，就可以得到文法的不同句型（可能既包含非终结符号，又包含终结符号，也可以是空串）。

例子

- 文法：$E \rightarrow -E \mid E+E \mid E*E \mid (E) \mid \text{id}$
- 推导序列：$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(\text{id})$

所有能推导出的内容就是文法定义的语言。

文法 $G = (V, \Sigma, R, S)$ 的语言是集合

$$L(G) = \{w \in \Sigma^* : S \overset{*}{\Rightarrow} w\}$$

注意区分句型和语言。

（3）语法分析树

一棵语法分析树可对应多个推导序列，但只有唯一的最左推导及最右推导（ 最左推导:每一步替换最左边的非终结符；最右推导:每一步替换最右边的非终结符 最右推导称为规范推导）。

- $E \Rightarrow E+E \Rightarrow id+E \Rightarrow id+E*E \Rightarrow id+id*E$
  $\Rightarrow id+id*id$
- $E \Rightarrow E*E \Rightarrow E+E*E \Rightarrow id+E*E \Rightarrow id+id*E$
  $\Rightarrow id+id*id$

都是最左推导

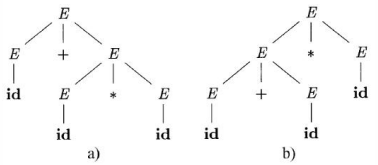


图 4-5　id + id * id 的两棵语法树

由推导式构造语法分析树的算法– 初始化：a1 的分析树是标号为 A 的单个结点– 假设已经构造出 ai-1= X1X2…Xk 的分析树，且 ai-1 到 ai 的 推导是将 Xj 替换为β，那么在当前分析树中找出第 j 个非 ε结点，向这个结点增加构成β的子结点；如果β= ε， 则增加一个标号为ε的子结点

（4）下推自动机

可作为考虑 CFG 的一种思维方式。

下推自动机 – 维基百科，自由的百科全书

进入正题

书中主要有两种语法分析的方式。

自顶向下语法分析器(通常用于处理 LL 文法)– 从语法分析树的根部开始构造语法分析树。

自底向上语法分析器(通常用于处理 LR 文法)– 从语法分析树的叶子开始构造语法分析树。

两种语法分析只能处理特定类型的文法，但足以描述程序设计语言。

而 LL（1）算法则是一种自顶向上的语法分析算法。

它可以缓解选择产生式的无意义回溯问题，具有一定预测能力。对于 LL(1)文法，可以在自顶向下分析过程中，根据当前输入符号来确定使用的产生式。

递归下降的语法分析

- 每个非终结符号对应于一个过程，该过程负责扫描此非终结符号对应的结构
- 程序执行从开始符号对应的过程开始
  - 当扫描整个输入串时宣布分析成功完成

```
    void A() {
1)      选择一个 A 产生式，A → X1X2 ··· Xk;
2)      for ( i = 1 to k ) {
3)          if ( Xi 是一个非终结符号 )
4)              调用过程Xi();
5)          else if ( Xi 等于当前的输入符号a )
6)              读入下一个输入符号;
7)          else /* 发生了一个错误 */;
        }
    }
```

步骤为：

（1）可能需要先消除左递归和提取左公因子（取决于题目要求，因为具有左递归或是左公因子的一定不是 LL(1)的）

（2）计算每个非终结符的 FIRST 和 FOLLOW

（3）构建预测表（对于文法 G 的每个产生式 A→ α ；对于 FIRST(α)中的每个终

结符号 a，将 A→ α 加入到 M[A, a]中；如果 ε 在 FIRST(α)，那么对于 FOLLOW(A)中的每个符号 b，将 A→ α 也加入到 M[A, b]中 – 最后在所有的空白条目中填入 error）

（4）从左到右扫描序列，根据预测表决定所使用的推导法则，如下图，其实就是下推自动机的思想。

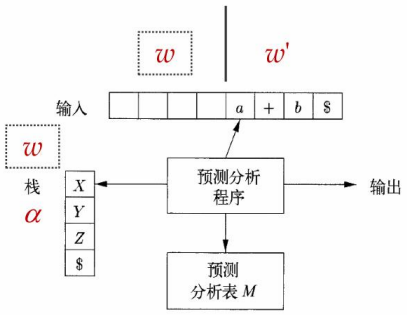


图 4-19　一个分析表驱动的预测分析器的模型

- 输入：id + id * id

非终结符号	输入符号					
	id	+	*	(	)	$
E	$E \to TE'$			$E \to TE'$		
E'		$E' \to +TE'$			$E' \to \epsilon$	$E' \to \epsilon$
T	$T \to FT'$			$T \to FT'$		
T'		$T' \to \epsilon$	$T' \to *FT'$		$T' \to \epsilon$	$T' \to \epsilon$
F	$F \to$ id			$F \to (E)$		

已匹配	栈	输入	动作	
	$E\$$	id + id * id$		
	$TE'\$$	id + id * id$	输出	$E \to TE'$
	$FT'E'\$$	id + id * id$	输出	$T \to FT'$
	id $T'E'\$$	id + id * id$	输出	$F \to$ id
id	$T'E'\$$	+ id * id$	匹配	id
id	$E'\$$	+ id * id$	输出	$T' \to \epsilon$
id	$+ TE'\$$	+ id * id$	输出	$E' \to + TE'$
id +	$TE'\$$	id * id$	匹配	+

已匹配的部分加上栈中的符号必然是一个最左句型

下面详细介绍左递归消除、提取左公因子及 LL(1)算法。

（1）消除左递归（书 4.3.3）

什么是左递归？

如果一个文法中有非终结符号A使得$A \overset{+}{\Rightarrow} A\alpha$，那么这个文法就是左递归的

立即左递归 – 文法中存在一个形如 A→A α 的产生式

自顶向下的语法分析技术不能处理左递归的情况， 因此需要消除左递归，但是自底向上的技术可以处理左递归。

先从立即左递归的消除讲起。

假设非终结符号A存在立即左递归的情形

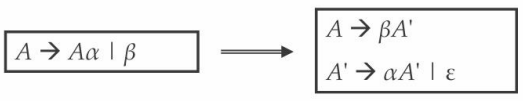
$A \to A\alpha_1 \mid \ldots \mid A\alpha_m \mid \beta_1 \mid \ldots \mid \beta_n$

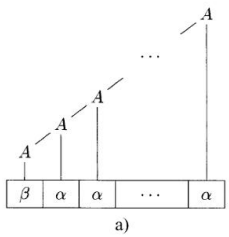
可替换为

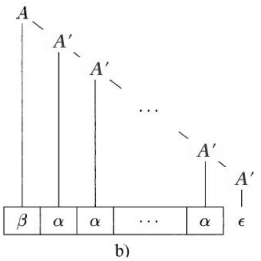
$A \to \beta_1 A' \mid \ldots \mid \beta_n A'$

$A' \to \alpha_1 A' \mid \ldots \mid \alpha_m A' \mid \varepsilon$

观察：由A生成的串以某个β_i开头，然后跟上零个或多个α_j







$$
\begin{array}{ll}
 & E \rightarrow TE' \\
E \rightarrow E + T \mid T & E' \rightarrow + \ TE' \mid \epsilon \\
T \rightarrow T * F \mid F \quad \Longrightarrow & T \rightarrow FT' \\
F \rightarrow (E) \mid \mathbf{id} & T' \rightarrow * \ FT' \mid \epsilon \\
 & F \rightarrow (\ E\ ) \mid \mathbf{id}
\end{array}
$$

再考虑消除多步左递归

```
1)  按照某个顺序将非终结符号排序为 A1, A2, ..., An.
2)  for ( 从1到n的每个 i ) {
3)      for ( 从1到 i − 1的每个 j ) {
4)          将每个形如 Ai → Ajγ 的产生式替换为产生式组 Ai → δ1γ | δ2γ | ··· | δkγ,
            其中 Aj → δ1 | δ2 | ··· | δk 是所有的Aj产生式
5)      }
6)      消除 Ai 产生式之间的立即左递归
7)  }
```

图4-11　消除文法中的左递归的算法

图4-11中的过程的工作原理如下。在i=1的第一次迭代中，第2～7行的外层循环消除了A_1产生式之间的所有立即左递归。因此，余下的所有形如$A_1 \rightarrow A_l\alpha$的产生式都一定满足l>1。在外层循环的第i-1次迭代之后，所有的非终结符号A_k（k<i）都被“清洗”过了。也就是说，任何产生式$A_k \rightarrow A_l\alpha$都必然满足l>k。结果，在第i次迭代中，第3～5行的内层循环不断提高所有形如$A_i \rightarrow A_m\alpha$的产生式中m的下界，直到m≥i成立为止。然后，第6行消除了A_i产生式中的立即左递归，保证m>i成立。

最后还有一步要记得消除死状态。

但其实（如书例 4.20）有些有伊普西隆产生式也能得到正确结果。

（通用左递归消除有一定问题，在于很难找到新文法和旧 文法推导之间的对应关系，很难进行语义处理）。

（2）提取左公因子（书 4.3.4）

输入：文法G。

输出：一个等价的提取了左公因子的文法。

方法：对于每个非终结符号A，找出它的两个或多个选项之间的最长公共前缀α。如果$\alpha \neq \in$，即存在一个非平凡的公共前缀，那么将所有A产生式$A \rightarrow \alpha\beta_1 | \alpha\beta_2 | \ldots | \alpha\beta_n | \gamma$，替换为

$A \rightarrow \alpha A' | \gamma$

$A' \rightarrow \beta_1 | \beta_2 | \ldots | \beta_n$

其中，γ表示所有不以α开头的产生式体；A′是一个新的非终结符号。不断应用这个转换，直到每个非终结符号的任意两个产生式体都没有公共前缀为止。

（3）FIRST 和 FOLLOW（书 4.4.2）

在自顶向下的分析技术中，使用向前看几个符号来确定产生式(通常只看一个符号)

当前句型是$xA\beta$，而输入是$xa\ldots$，那么选择产生式$A \rightarrow \alpha$的必要条件是下列之一

- $\alpha \overset{*}{\Rightarrow} a\ldots$
- $\alpha \overset{*}{\Rightarrow} \varepsilon$，且$\beta$以a开头，即在某个句型中a跟在$A$之后
- 如果按照这两个条件选择时能够保证唯一性，那么我们就可以避免回溯

因此，我们定义FIRST和FOLLOW

FIRST(α) - 可以从α推导得到串的首符号的集合

FIRST 函数的意义 - A 的产生式 A→α | β，且 FIRST(α)和 FIRST(β)不相交 - 下一个输入符号是 a，若 a∈FIRST(α)，则选择 A→α ，若 a∈FIRST(β)，则选择 A→β 。这样就具有了选择的确定性。

FIRST的计算方法

- 计算FIRST(X)
  - X是终结符号，那么加入X
  - X是非终结符号，且$X \rightarrow Y_1Y_2\ldots Y_k$是产生式
    - 如果a在FIRST(Y_i)中，且ε在FIRST(Y_1), … , FIRST(Y_{i-1})中，那么也加入a
    - 如果ε在FIRST(Y_1), … , FIRST(Y_k)中，那么也加入ε
  - X是非终结符号且$X \rightarrow \varepsilon$，那么也加入ε
- 计算FIRST($X_1X_2\ldots X_n$)
  - 加入FIRST(X_1)中所有非ε符号
  - 若ε在FIRST(X_1)中，加入FIRST(X_2)中所有非ε符号 …
  - 若ε在所有FIRST(X_i)中，也加入ε

FOLLOW(A) - 可能在某些句型中紧跟在 A 右边的终结符号的集合 - 例如：S→αAaβ，终结符号 a ∈FOLLOW(A)

FOLLOW 函数的意义 - 如果 A→α，当α→ 或α=> 时，FOLLOW(A)可以帮 助我们选择恰当的产生式 - 例如：A→α，而 b 属于 FOLLOW(A)，如果α=> ，而 当前输入符号是 b，则可以选择 A→α，因为 A 最终到 达了 ，而且后面跟着 b。

算法

- 将**右端结束标记**$加入FOLLOW($S$)中
- 按照下面两个规则不断迭代，直到所有的FOLLOW集合都不再增长为止
  - 如果存在产生式$A \rightarrow \alpha B\beta$，那么FIRST($\beta$)中所有非ε的符号都加入FOLLOW($B$)中
  - 如果存在一个产生式$A \rightarrow \alpha B$，或者$A \rightarrow \alpha B\beta$且FIRST($\beta$)包含ε，那么FOLLOW($A$)中所有符号都加入FOLLOW($B$)中

（4）预测表的构建

方法

- 对于文法G的每个产生式$A \rightarrow \alpha$
  - 对于FIRST(α)中的每个终结符号a，将$A \rightarrow \alpha$加入到$M[A, a]$中
  - 如果ε在FIRST(α)，那么对于FOLLOW(A)中的每个符号b，将$A \rightarrow \alpha$也加入到$M[A, b]$中
- 最后在所有的空白条目中填入error

（5）整体流程

LL(1)文法的递归下降分析

- 递归下降语法分析程序由一组过程组成
- 每个非终结符号对应于一个过程，该过程负责扫描该非终结符号对应的结构
- 可以使用当前的输入符号来唯一地选择产生式

```
    void A() {
1)      选择一个 A 产生式，A → X1X2 ··· Xk;
2)      for ( i = 1 to k ) {
3)          if ( Xi 是一个非终结符号 )
4)              调用过程Xi();
5)          else if ( Xi 等于当前的输入符号a )
6)              读入下一个输入符号;
7)          else /* 发生了一个错误 */;
        }
    }
```

如果当前输入符号为a，那么选择$M[A, a]$中的产生式

非递归下降分析

- 输入：串w，预测分析表M
- 输出：如果w是句子，输出w的最左推导；否则报错

(1) 初始化：输入缓冲区中为w\$，栈中为S\$，ip指向w的第一个符号

(2) 令X=栈顶符号，ip指向输入符号a

if ($X == a$) X出栈，ip向前移动 // 与终结符号匹配成功

else if (X是终结符号) error() // 失配

else if ($M[X, a]$是报错条目) error() // 无适当的产生式

else if ($M[X, a] = X \rightarrow Y_1Y_2 \ldots Y_k$) {

输出产生式$X \rightarrow Y_1Y_2 \ldots Y_k$

弹出栈顶符号X，并将$Y_k, Y_{k-1}, \ldots, Y_1$压入栈中

}

(3) 不断执行第二步，直到要么报错，要么栈中为空

回过头再看 LL(1)文法的定义。

定义：对文法的任意两个产生式$A \rightarrow \alpha \mid \beta$

- 不存在终结符号a使得α和β都可推导出以a开头的串
- α和β最多只有一个可推导出空串
- 如果β可推导出空串，那么α不能推导出以FOLLOW(A)中任何终结符号开头的串

等价于

- FIRST(α) ∩ FIRST(β) = Φ (条件一、二)
- 如果ε ∈ FIRST(β)，那么FIRST(α) ∩ FOLLOW(A) = Φ; 反之亦然 (条件三)

（补：二义性的消除，了解即可）

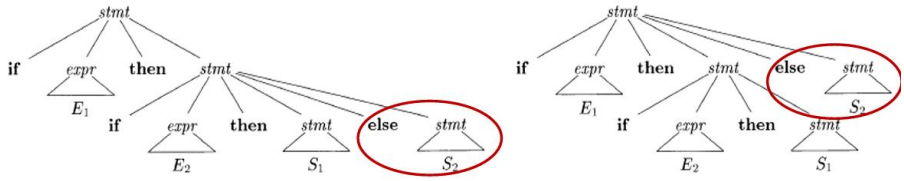
- 一些二义性文法可被改成等价的无二义性的文法
- 例子

  stmt → **if** *expr* **then** *stmt*

  | **if** *expr* **then** *stmt* **else** *stmt*

  | **other**

  - **if** E_1 **then if** E_2 **then** S_1 **else** S_2的两棵语法树



- 保证else和最近未匹配的then匹配
  - 要求在then和else之间出现的语句必须是匹配好的
- 引入*matched_stmt*表示匹配好的语句

  stmt → *matched_stmt* | *open_stmt*

  matched_stmt →

  if *expr* **then** *matched_stmt* **else** *matched_stmt*

  | **other**

  open_stmt → **if** *expr* **then** *stmt*

  | **if** *expr* **then** *matched_stmt* **else** *open_stmt*
- 二义性的消除方法没有规律可循